

Penny: project foundations

Document version: 1.0 · **Date:** September 29, 2026

This document defines the problem Penny addresses, proposes how the team can share Scrum responsibilities, and sets the initial scope boundaries. The people assignments are a **project proposal**; they do not claim that these individuals have already performed the listed work.

1. Background and real-world software problem

Everyday expenses are often recorded in scattered places or reconstructed from memory at the end of the week. Without a record of the amount, date, and context of each purchase, a person may know how much money remains but struggle to see which categories consumed it, which days had the most spending, or how small expenses add up.

Problem statement: a person who wants to manage personal spending needs a quick way to record each expense and a reliable weekly summary, without manually adding receipts, notes, or separate transactions. For someone who primarily uses a phone, recording should also work when a computer or server is not running.

Intended users: an individual managing their own expenses, including students and other people who want to understand weekly spending habits. The project does not assume access to bank accounts and is not a substitute for professional financial advice.

System response: Penny records a description, amount in Mexican pesos (MXN), date, and optional note; suggests a category or accepts a manual choice; presents a chart and expense list; and produces a Monday-to-Sunday report with daily totals and a PDF. The web version stores data in a local JSON file through Express. The Android application stores data in SQLite on the phone and works offline. The two stores are independent.

This problem description is a **design need identified for the project**, not a claim that a documented user survey or field study has been completed. Later user validation could measure entry time, usage frequency, and report clarity.

2. Scrum team roles and responsibilities

The formal Scrum accountabilities are **Product Owner**, **Scrum Master**, and **Developers**. The technical focuses below divide work among the Developers; they are not additional formal Scrum roles.

Team member	Scrum accountability	Proposed focus and decisions
David Portillo	Product Owner	Order the Product Backlog, define user value, clarify acceptance criteria, and decide product scope priorities with the team.
Edwin Ramirez	Scrum Master	Facilitate planning, reviews, and retrospectives; help remove impediments; promote transparent progress and effective use of Scrum.
Josue Perez	Developer, web interface and user experience	Implement forms, navigation, spending views, accessibility, and layouts for different screen sizes.
Christian Cisneros	Developer, API and business rules	Maintain Express routes and validation, the expense model, categories, web persistence, and weekly report calculations.
Abdel Gutierrez	Developer, Android and quality	Integrate Capacitor and SQLite, prepare backups and APK builds, and coordinate functional checks on Android.

Shared responsibility: every member takes part in Sprint planning, estimates and adjusts work, reviews increments, maintains quality, and collaborates when work crosses the proposed specializations. The Product Owner prioritizes the product; Developers decide how to perform the technical work. The team should agree on this distribution when work begins.

3. Initial scope boundaries and system exclusions

Initial prototype scope

The **initial baseline** was a local, single-user web prototype. It included:

- 1 Creating, editing, and deleting expenses with a description, positive amount, date, category, and optional note.
- 2 Suggesting a category from the description while allowing manual selection.
- 3 Keeping records across sessions in the local server's JSON file.
- 4 Showing expenses for a selected week, with list search, totals, and a category chart proportional to spending.
- 5 Viewing a Monday-to-Sunday report, including days without expenses, and downloading a PDF on demand.
- 6 Validating data before saving and calculating money with integer cents.

Operating boundary: one user and one local server process; the browser must reach Express to use the web version. The initial scope did not include a public Internet-hosted service.

Initial scope exclusions

- User accounts, authentication, permissions, and simultaneous multi-user access.
- Automatic synchronization between devices, a cloud database, and web operation without the local server.
- Connections to banks, cards, or payment systems; automatic transaction imports.
- Budgets, income, debt tracking, forecasts, or financial recommendations.
- Currency conversion; the prototype handles one currency at a time. The current version displays MXN.
- Reminders, recurring expenses, and automatic Sunday report generation.
- App store publication and automatic updates.

Subsequent scope extension

An installable Android application was added after the initial web prototype. It reuses the interface but processes and stores expenses locally in SQLite, adds manual JSON backup and import, and can share the PDF. This extension **does not synchronize** the web and Android data. The current app displays amounts in MXN and preserves previously entered numeric values without conversion.

Basis for this description

The scope was checked against `client/src/App.jsx`, `client/src/api.js`, `client/src/localExpenseStore.js`, `client/src/reportPdf.js`, `server/src/routes/expenseRoutes.js`, `server/src/models/Expense.js`, and `server/src/storage/ExpenseStore.js`. The role assignments are an organizational proposal, not information obtained from the repository.